158.337 Database Development

Week 1 – Database Systems

The reading for the first week of the paper is Chapter 1 and Chapter 2 – Database Systems.

Chapter 1

You interact with databases on a daily basis. For example, how many of you have bought anything using a credit card during the past day, week, month, or year? How many have shipped a document or a package via an overnight service or via certified or registered mail? **None of those transactions would be possible without a database.** Databases are important because we depend on their existence to perform countless transactions and to provide information. It is important that the (database) transactions are made successfully, accurately, and timely. This leads to the importance of database design, which is your responsibility as a database designer.

In this paper you'll learn about database design and implementation. And, of course, databases don't manage themselves … and that point leads to the importance of the database administration (DBA) function (where you come in with more responsibilities).

Data vs. Information
Data are raw facts. Information is processed data to reveal the meaning behind the facts.

Database management system (**DBMS**) - serves as the intermediary between the user and the database. The DBMS receives all application requests and translates them into the complex operations required to fulfill those requests. The DBMS hides much of the database's internal complexity from the application programs and users. The application program might be written by a programmer using a programming language such as COBOL, Visual Basic, or C++, or it might be created through a DBMS utility program.

Why DBMS is important?
Modern database and applications development software is easy to use and many people can quickly learn to implement a simple database and develop simple applications. But as data and reporting requirements become more complex, those same people will simply (and quickly!) produce the required add-ons. This will result in unnecessarily duplicated data to be found in the database. That's how data redundancies and all their attendant anomalies develop, thus reducing the "database" and its applications to a status worse than useless.

A database created without the benefit of a detailed blueprint is unlikely to be satisfactory.
Would you think it smart to build a house without the benefit of a blueprint? Perhaps it would be OK to build a chicken coop without a blueprint, but would you want your house to be built the same way?

Is spreadsheet the same as database?
Spreadsheets do not support self-documentation through metadata, i.e. definitions of data and their relationships, enforcement of data types or domains to ensure consistency of data within a column, defined relationships among tables, or constraints to ensure consistency of data across related tables.

A collection of spreadsheets has several problems:

1. End users create their own, private, copies of the data, which creates issues of data ownership. This situation also creates islands of information where changes to one set of data are not reflected in all of the copies of the data. This leads to the second problem.
2. Lack of data consistency. Because the data in various spreadsheets may be intended to represent a view of the business environment, a lack of consistency in the data may lead to faulty decision making based on inaccurate data.

There is a world of exciting database employment opportunities out there. Database technologies are constantly evolving and to take advantage of new opportunities, you must be able to apply good skills to understanding design issues and answer questions, such as

- How databases have evolved from manual file systems? What challenges did we face earlier?
- What are the effects of good and poorly designed databases?
- What is the use of metadata?
- Can we totally do away with data redundancy?
- How can structural and data dependence be achieved?
- What are the advantages/ disadvantages of file sharing?
- What are the new emerging databases? We can see that Facebook/ Twitter, etc. breaking the traditional way of storing/retreiving data – so how is that happening?
- Etc., etc., etc., …….

Chapter 2

The reading for the second week of the paper is Chapter 2 – Data Models. This chapter will explain how business rules are translated into data modelling concepts for a given problem domain.

Data modeling
A data model is a relatively simple representation, usually graphical, of a more complex real world object event. The data model's main function is to help us understand the complexities of the real-world environment. The database designer uses data models to facilitate the interaction among designers, application programmers, and end users, so each gets an understanding of the underlying business rules.

Business rules are precisely written and are unambiguous statements that are derived from a detailed description of an organization's operations. ***When written properly***, business rules define the modeling components: entities, relationships, attributes, connectivities, cardinalities and constraints.

Examples of business rules are:

- A store employs many employees.
- Each employee is employed by only one store.

- A college has many departments.
- Each department belongs to a single college. (This business rule reflects a university that has multiple colleges such as Business, Liberal Arts, Education, Engineering, etc.)

Relationship definition requires the definition of two business rules. This two-way requirement exists because there is always a two-way relationship between any two related entities. But, also keep in mind that the ER diagrams cannot always reflect all of the business rules. For example, examine the following business rule:

> A customer cannot be given a credit line over $10,000 unless that customer has maintained a satisfactory credit history (as determined by the credit manager) during the past two years.

This business rule describes a constraint that cannot be shown in the ER diagram. The business rule reflected in this constraint would be handled at the applications software level through the use of a trigger or a stored procedure (which we will learn later when we do PL/SQL).

So, how do you translate business rules into data model components? As a general rule, a noun in a business rule will translate into an entity in the model, and a verb (active or passive) associating nouns will translate into a relationship among the entities. For example, the business rule "A store employs many employees" contains two nouns (store and employee) and a verb ("employs") that associates them.

Relational Data Model

The relational database model is perceived by the user to be a collection of tables in which data are stored. Each table resembles a matrix composed of row and columns. Tables are related to each other by sharing a common value in one of their columns.

Entity relationship diagram (ERD) representation:
"A customer can make many payments, but each payment is made by only one customer"



Database modelers use the term *connectivity* to label the minimum and maximum number of entity occurrences for each entity in a relationship. Connectivities for the 1:1 relationship is (1,1), for the 1:M relationship it is (1,M), and for the M:N relationship it is (M,N). The Crow's Foot ERD indicates connectivity through the use of symbols. The "one" side of a relationship is indicated by a short line segment placed next to the entity box. The "many" side of a relationship is indicated by a three-pronged symbol resembling a bird's foot placed next to the entity box.

Relational DBMS ensures data and structural independence. Further using a declarative language, i.e. Structured Query Language (SQL), we can access data as needed. For example to get a listing of all the customers whose last name is "Smith" in a relational database, the statement used is:

```
SELECT       *
FROM    CUSTOMER
WHERE    CUS_LNAME = "Smith";
```

Data modelling requirements describe three levels of data abstraction at external, conceptual and internal levels and are laid down by the American National Standards Institute Standards Planning and Requirements Committe (ANSI/SPARC). Recently, this has been expanded to a fourth level of abastraction called physical level.

- The **external model** is the end users' view of the data environment.
- The **conceptual model** represents a global view of the entire database by the entire organization.
- The **internal model** is the representation of the database as "seen" by the DBMS. In other words, the internal model requires the designer to match the conceptual model's characteristics and constraints to those of the selected implementation model.
- The **physical model** operates at the lowest level of abstraction, describing the physical storage device and (physical) access methods..

Recent times

….. a new wave of data has "emerged", which has triggered a phenomenon called "Big Data". These data are characterized for being high volume (petabyte size and beyond), high frequency (data are generated almost constantly), and mostly semi-structured. It is labelled via "3V" framework.

- **V**olume: Refers to the amounts of data being stored. With the adoption and growth of the Internet and social media, companies have multiplied the ways to reach customers. This ever-growing volume of data quickly reached petabytes in size and it's still growing.
- **V**elocity: Refers not only to the speed with which data grows but also to the need to process these data quickly in order to generate information and insight. The velocity of data growth is due to the increase in the number of different data streams from which data is being piped to organizations (via the web, e-commerce, Tweets, Facebook posts, emails, sensors, GPS, and so on).
- **V**ariety: Refers to the fact that the data being collected comes in multiple different data formats. A great portion of these data comes in formats not suitable to be handled by the typical operational databases based on the relational model.

Every time you search for a product on Amazon, send messages to friends in Facebook, watch a video in YouTube or search for directions in Google Maps, you are using a NoSQL database. **NoSQL** refers to a new generation of databases that address the very specific challenges of the "big data" era. This data is not based on the relational model, hence no SQL is required. So, you can guess how the name NoSQL transpired. It is based on a key-value model.

NOTE: We will cover concepts related to key-value models later when we use NoSQL databases.

We will learn to model different problem domains using CASE tools (e.g., Visio, JDeveloper), create tables in the Oracle 11g platform, write SQL statements and triggers, and get a grip of database functionalities using latest technologies.

Note: You may either use Visio (from Massey via DreamSpark) or JDeveloper (from http://www.oracle.com) for drawing database models. With JDeveloper, you can write queries (similar to sqldeveloper) and model entities (similar to Visio).

Interesting times lie ahead!!

Some database jokes (picked up from the Internet)

- A big party is being held to honor relational database systems and their impact on modern society. Outside the venue, the host awaits the guests.

  The first limousine arrives and out steps Oracle followed by 4 people.
  Host: Who have you brought along?
  Oracle: I have 4 DBA's in tow. One to install me, one to design the databases, one to administer me, and the other to justify the cost.

  A second limo arrives and out steps DB2 followed by 40 people.
  Host: Who have you bought along?
  DB2: I have 2 DBA's, 2 hardware specialists, and 36 consultants.

  A third limo arrives and out steps SQL Server all on his own.
  Host: Why haven't you brought anyone?
  SQL Server: I didn't bring anyone because I am easy to install and am basically self managing. But I did bring the #sqlhelp Twitter hashtag for when the excrement hits the fan.

  20 minutes later, up rushes MySQL, unshaved, hair a mess.
  Host: Where have you been MySQL?
  MySQL: Sorry, I thought it was February 31st.

- What did the baby SQL say to its father?
  daaaaaa-ta ?

- An IT contractor goes to the gates of Heaven.
  Very indignantly, he says to St Peter, "Look here, I was only 45 so why did I have to die? It's not fair."
  St Peter stares back with a puzzled frown, and leafs through the golden book. "Hey, that's odd. According to the query result, I can see hours you've claimed for, you're 120."

Cheers,
Anu
a.s.mathrani@massey.ac.nz